



AZ-204

Day 5: Security, Identity & Observability

Azure Developer Associate Training — Day 5 of 5

AZ-204

DAY 5

Day 5 Agenda

Today we cover the final — and most critical — pillars of production-ready Azure development.

01

Azure Security for Developers

Managed Identity, Key Vault, secrets management

02

Authentication & Authorization

OAuth 2.0, tokens, Microsoft Identity Platform

03

Monitoring & Observability

Application Insights, Log Analytics, distributed tracing

04

Real-World Scenarios & Labs

Secure API access, secret rotation, debugging at scale

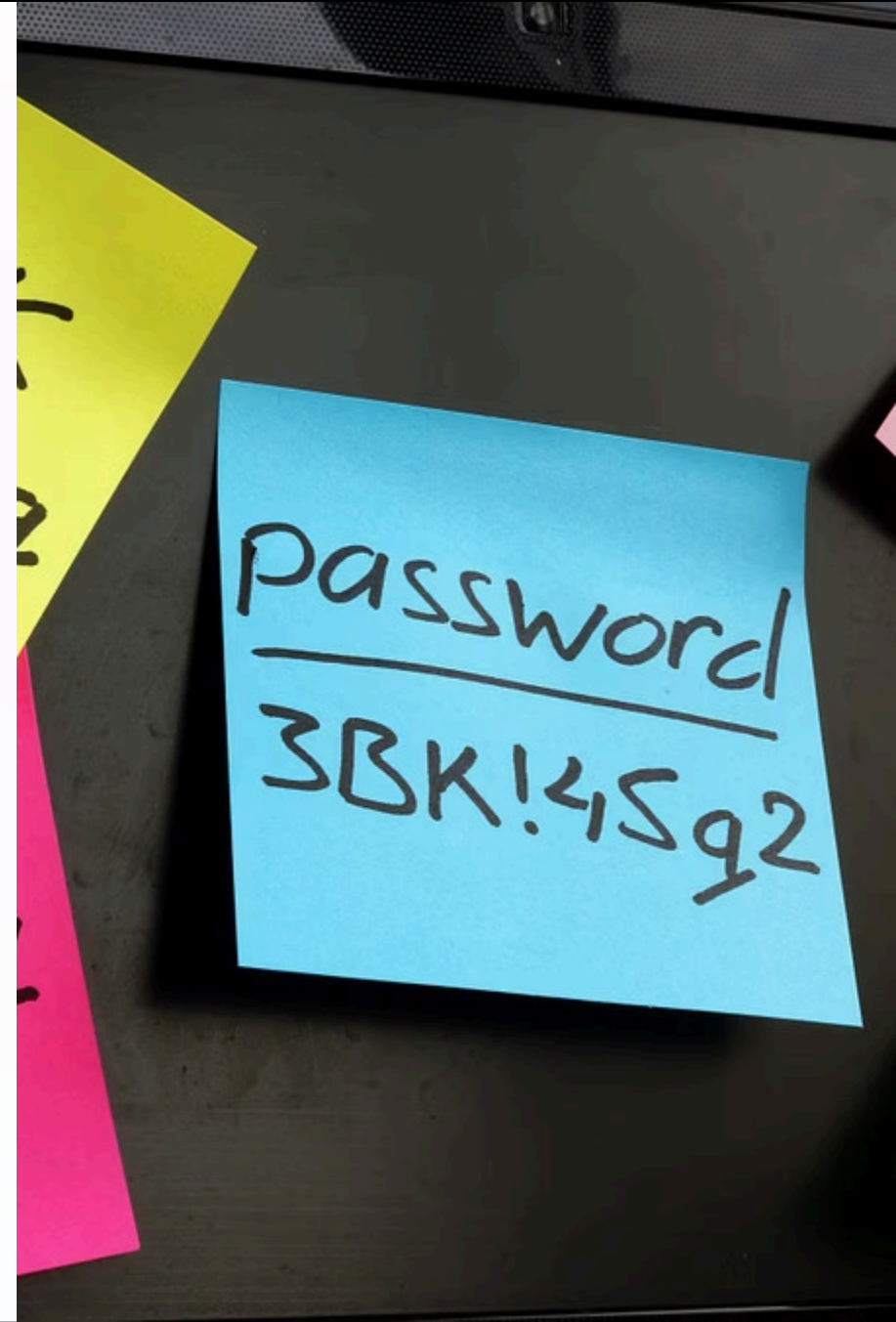
The #1 Developer Security Mistake

❌ Hardcoded Secrets

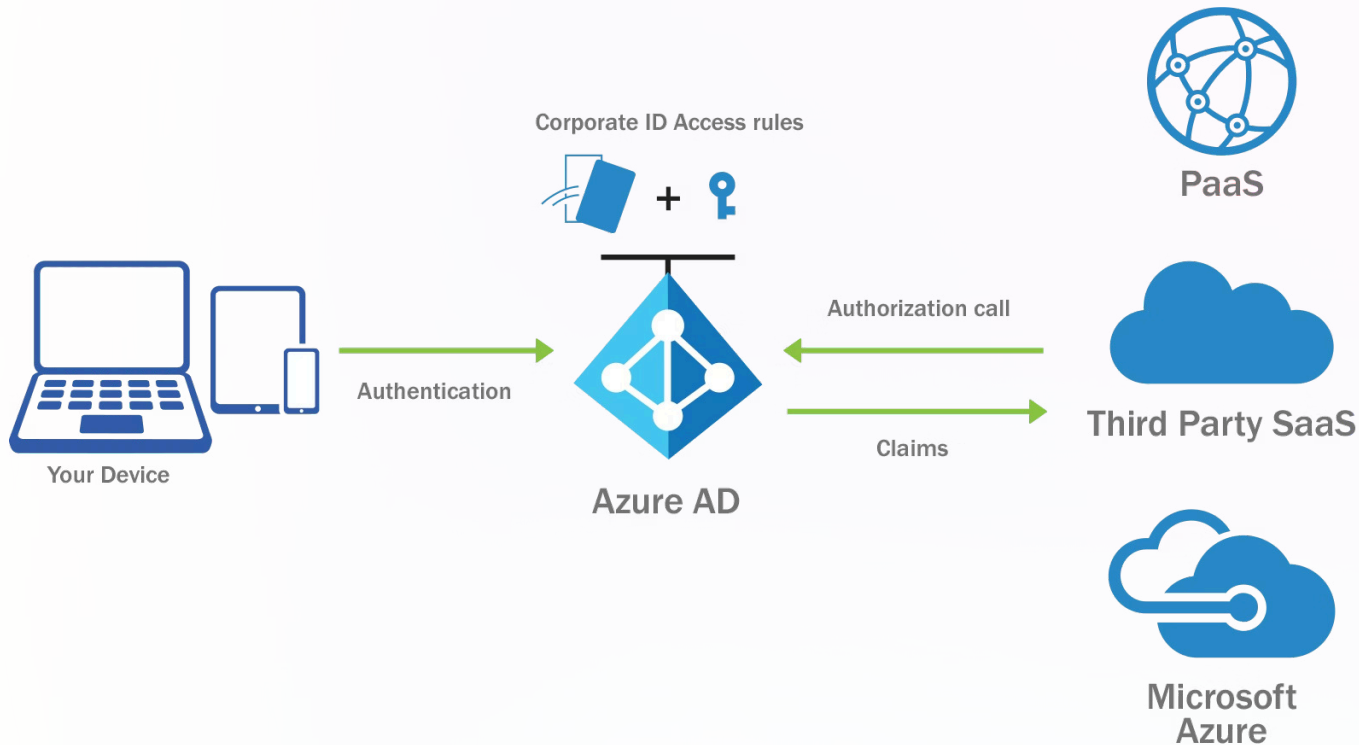
Connection strings, API keys, and passwords baked into source code — the most common and dangerous anti-pattern in production apps.

✅ The Azure Solution

Use **Managed Identity** + **Azure Key Vault** to eliminate secrets from code entirely. Zero credentials in config files, repos, or environment variables.



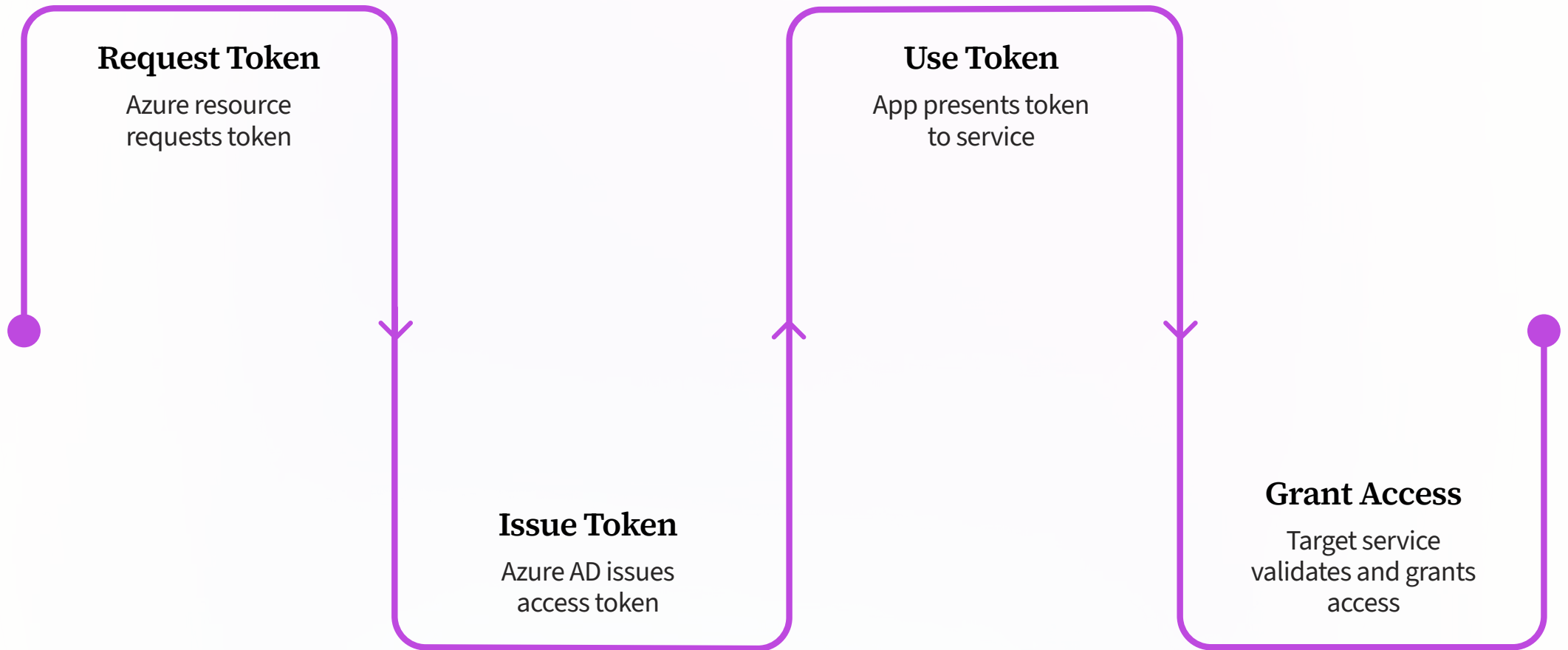
What Is Managed Identity?



A **Managed Identity** is an automatically managed identity in Azure Active Directory assigned to your Azure resource (e.g., App Service, Function App, VM).

- **System-assigned:** Tied to the lifecycle of one specific resource. Deleted when the resource is deleted.
- **User-assigned:** Created independently and can be shared across multiple resources.
- No passwords, no secrets, no certificates to manage — Azure handles rotation automatically.

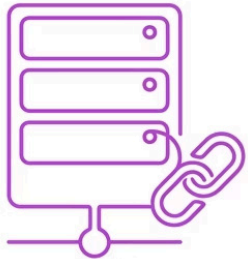
How Managed Identity Works



The application never sees a credential. Azure AD brokers trust between the resource and the target service using short-lived tokens — making this pattern both secure and operationally simple.

System-Assigned vs. User-Assigned Identity

System-Assigned Identity

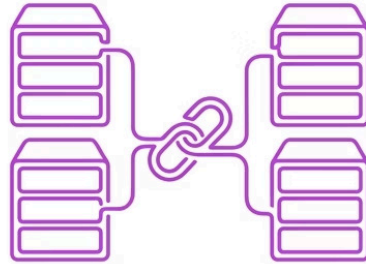


One identity per resource

Auto-deleted with resource

Simple setup

User-Assigned Identity



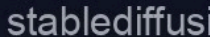
Shared across multiple resources

Independent lifecycle

Flexible reuse

When to Use Which

- Use **system-assigned** for single-resource, simple scenarios
- Use **user-assigned** when multiple apps share the same identity or permissions
- Both work seamlessly with Key Vault, Storage, SQL, and other Azure services



Azure Key Vault

A centralized, cloud-hosted secret store for managing **secrets**, **encryption keys**, and **certificates** — with full audit logging and RBAC access control.

What Key Vault Stores



Secrets

Connection strings, API keys, passwords. Versioned, with expiry and rotation support.



Keys

Cryptographic keys for encrypt/decrypt operations. Backed by HSM (Hardware Security Module) tiers.



Certificates

SSL/TLS certificates with auto-renewal. Integrates with App Service and API Management.

- ❏ Key Vault uses **soft-delete** by default — deleted secrets are retained for 90 days and can be recovered, preventing accidental data loss.

Accessing Key Vault from Code

C# — DefaultAzureCredential Pattern

```
var client = new SecretClient(  
    new Uri("https://myvault.vault.azure.net/"),  
    new DefaultAzureCredential());  
  
var secret = await  
    client.GetSecretAsync("DbConnectionString");  
string connString = secret.Value.Value;
```

DefaultAzureCredential automatically resolves the identity — Managed Identity in Azure, developer credentials locally. No code changes between environments.

Key Points

- Install `Azure.Security.KeyVault.Secrets` NuGet package
- Grant the app's Managed Identity **Key Vault Secrets User** role
- Works identically in App Service, Functions, and AKS
- No client secrets or certificates needed in code

Secrets Management Best Practices

Never Store Secrets in Source Code or Config Files

Use Key Vault references in App Service configuration — Azure resolves them at runtime without your app touching raw credentials.

Enable Secret Versioning and Rotation

Key Vault supports automatic rotation for supported services (Storage, SQL). Use versioned URIs to roll back if a rotation breaks something.

Restrict Access with RBAC, Not Access Policies

Azure RBAC on Key Vault is the modern recommended approach. Assign the least-privilege role needed — *Secrets User*, not *Secrets Officer*.

Enable Diagnostic Logging

Send Key Vault audit logs to Log Analytics. Every read, write, and delete is recorded — critical for compliance and incident response.

Real-World Scenario: Secure API Access



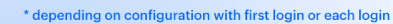
Production Pattern

An App Service calls a third-party payment API that requires a secret API key. Instead of storing the key in `appsettings.json`:

1. Enable system-assigned identity on App Service
2. Store the API key as a secret in Key Vault
3. Grant the identity **Secrets User** role on the vault
4. App fetches the secret at startup using SDK — no human ever handles the key

AUTHENTICATION

Understanding **OAuth 2.0**, token-based access, and the Microsoft Identity Platform — the backbone of secure Azure application development.



OAuth 2.0 Core Concepts



Resource Owner

The user or service that owns the protected data and grants access permissions.



Client

The application requesting access on behalf of the resource owner (your app).



Authorization Server

Issues tokens after verifying identity. In Azure: **Microsoft Identity Platform** (Azure AD).



Resource Server

The API or service being protected. Validates tokens before serving requests.

Token-Based Access: Access vs. Refresh Tokens

Access Token



SHORT-LIVED (60-90 min)

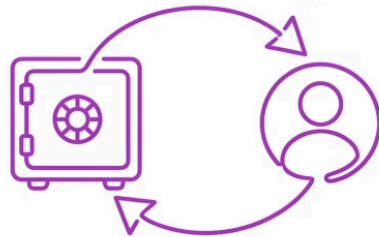
SENT WITH ALL API REQUESTS

JWT FORMAT

CONTAINS CLAIMS & SCOPES

STATELESS VALIDATION

Refresh Token



LONG-LIVED (days/weeks)

STORED SECURELY

GETS NEW ACCESS TOKENS

AVOIDS RE-LOGIN

NEVER SENT TO APIs

In Practice

- Access tokens are **JWTs** — decode them at *jwt.ms* to inspect claims during debugging
- Never store access tokens in `localStorage` — use secure, `HttpOnly` cookies or in-memory
- APIs validate tokens by checking signature and `aud` (audience) claim
- Expired tokens return HTTP **401** — clients must refresh silently

Microsoft Identity Platform & MSAL

What Is It?

Azure AD's OAuth 2.0 and OpenID Connect endpoint. The single identity provider for all Azure and Microsoft 365 services.

MSAL Library

Microsoft Authentication Library. Available for .NET, JavaScript, Python, Java, Go. Handles token acquisition, caching, and silent refresh automatically.

App Registration

Every app authenticating with Azure AD must be registered — defines client ID, redirect URIs, API permissions, and scopes.



Exam tip: Know the difference between **delegated permissions** (user context) and **application permissions** (app-only, requires admin consent).

Common OAuth 2.0 Grant Flows

1

Authorization Code + PKCE

For web apps and SPAs with a logged-in user. The most secure interactive flow. PKCE replaces client secret for public clients.

2

Client Credentials

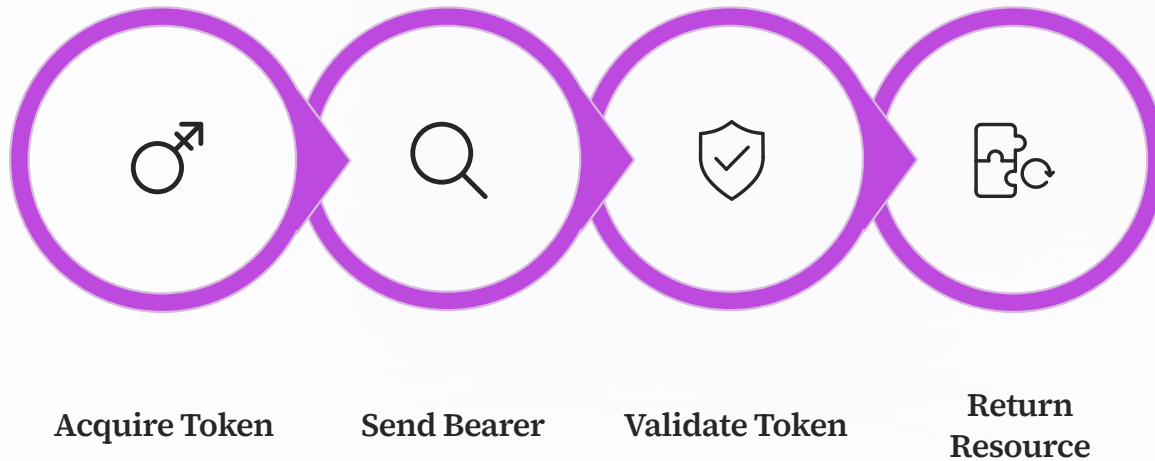
For daemon services and background jobs — no user involved. App authenticates with its own identity using a secret or certificate.

3

On-Behalf-Of (OBO)

Middle-tier APIs call downstream APIs while preserving the original user's identity and permissions. Common in microservice architectures.

Securing Your API with Azure AD



ASP.NET Core Implementation

- Add `Microsoft.Identity.Web` NuGet package
- Configure `AddMicrosoftIdentityWebApiAuthentication()` in `Program.cs`
- Decorate controllers with `[Authorize]` and scope checks
- Validate `aud`, `iss`, and custom claims in middleware

Token validation happens automatically — the middleware handles signature verification against Azure AD's public keys (JWKS endpoint).

CHAPTER 3

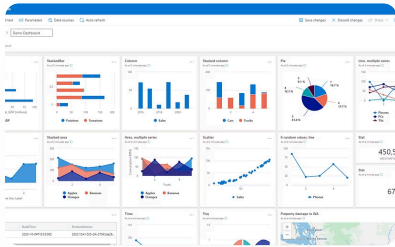
MONITORING & OBSERVABILITY

Monitoring & Observability on Azure

You can't fix what you can't see. Production systems require comprehensive telemetry — **Application Insights**, **Log Analytics**, and **distributed tracing** give you full visibility.

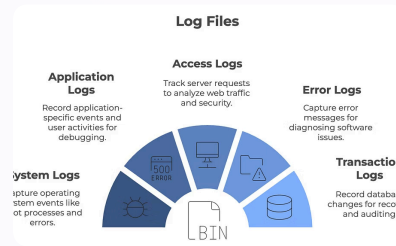


The Three Pillars of Observability



Metrics

Numeric time-series data — CPU, memory, request rate, failure rate. Used for alerting and autoscale triggers.



Logs

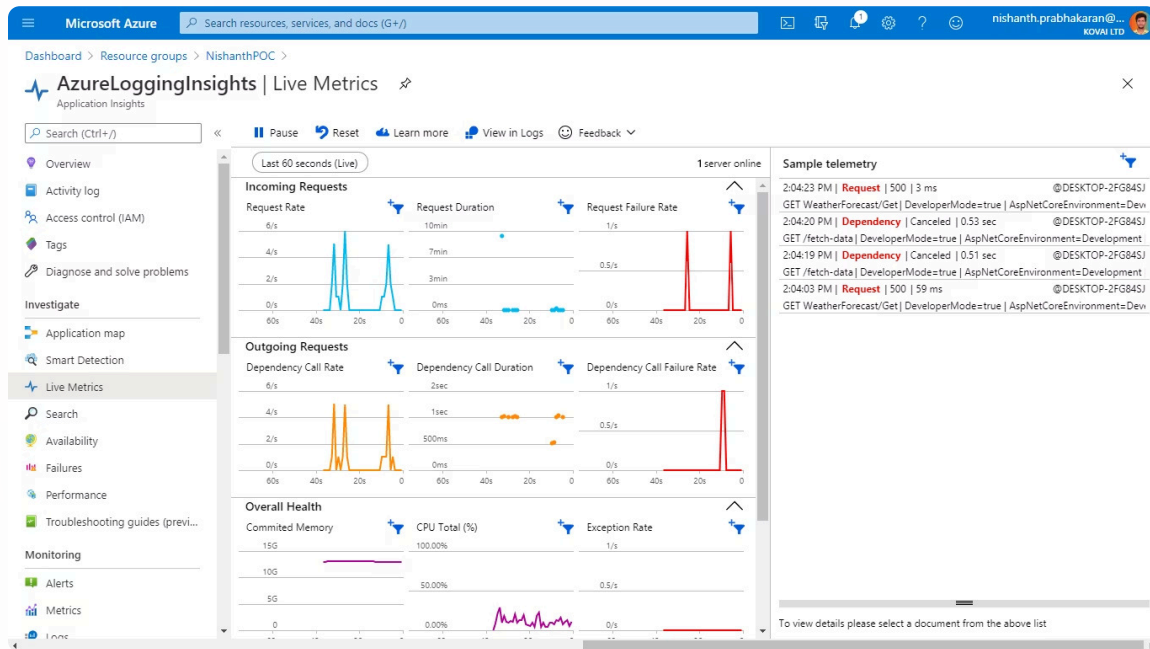
Structured or unstructured event records. Queryable with Kusto Query Language (KQL) in Log Analytics.



Traces

End-to-end request flows across services. Essential for debugging latency and failures in microservices.

Application Insights Deep Dive



What Application Insights Collects Automatically

- **Requests:** HTTP requests, response times, success rates
- **Dependencies:** Outbound calls to SQL, HTTP APIs, Storage
- **Exceptions:** Unhandled exceptions with full stack traces
- **Performance Counters:** CPU, memory, GC pressure
- **Custom Events & Metrics:** via `TelemetryClient`

Configure with a single

`APPLICATIONINSIGHTS_CONNECTION_STRING`

G environment variable — no instrumentation key needed for modern SDKs.

Log Analytics & KQL Essentials

Sample KQL Queries

```
// Failed requests in last hour
requests
| where timestamp > ago(1h)
| where success == false
| summarize count() by resultCode

// Top slow dependencies
dependencies
| where duration > 1000
| summarize avg(duration) by target
| order by avg_duration desc
```

Log Analytics Workspace

Central store for all Azure diagnostic logs, Application Insights data, and custom log sources.

- Query across multiple resources in one workspace
- Set **alerts** on KQL query results
- Build **Workbooks** for custom dashboards
- Retention: 30 days default, configurable up to 2 years

Distributed Tracing with Application Insights



Distributed tracing uses a shared **correlation ID** propagated via HTTP headers (traceparent / W3C Trace Context). Application Insights assembles spans into a full **end-to-end transaction** visible in the Application Map and Transaction Search blade.

Application Map & Live Metrics

Application Map

Topology view of all components and their dependencies. Red nodes indicate failure rates above threshold — instantly identify which service is causing cascading failures.

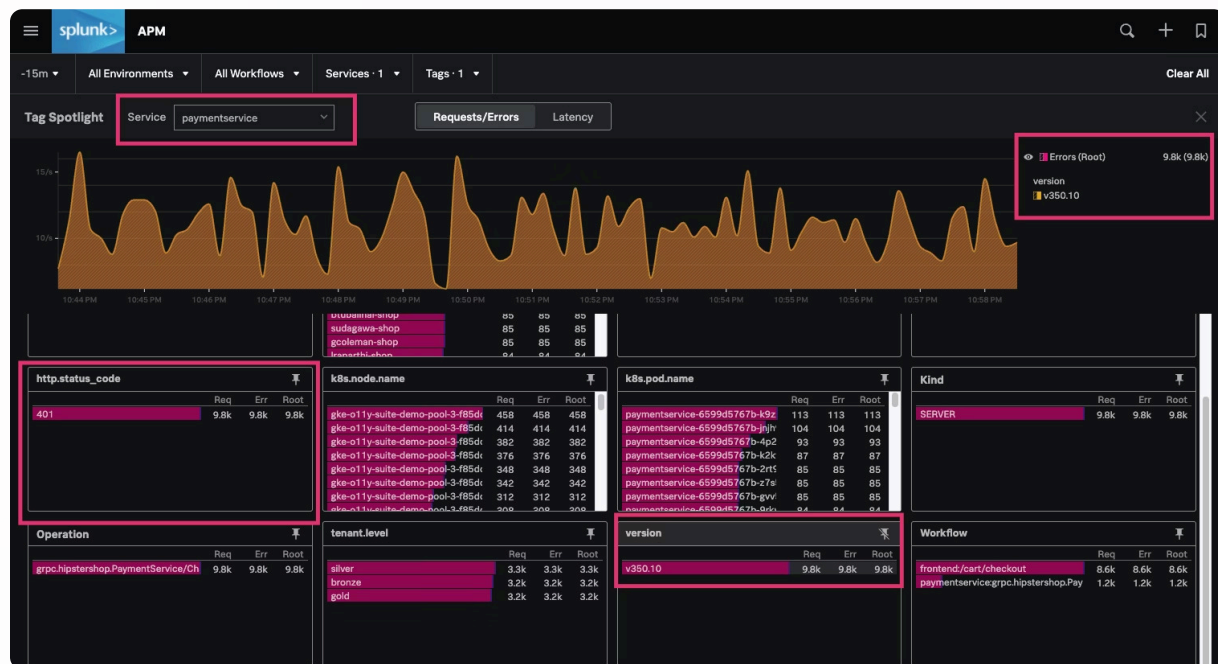
Live Metrics Stream

Sub-second telemetry with zero sampling. Use during deployments and incident response to see real-time request rates, failures, and server health without log latency.

Availability Tests

Synthetic ping and multi-step tests from Azure global locations. Alert immediately when your endpoint is unreachable from any region — before your users notice.

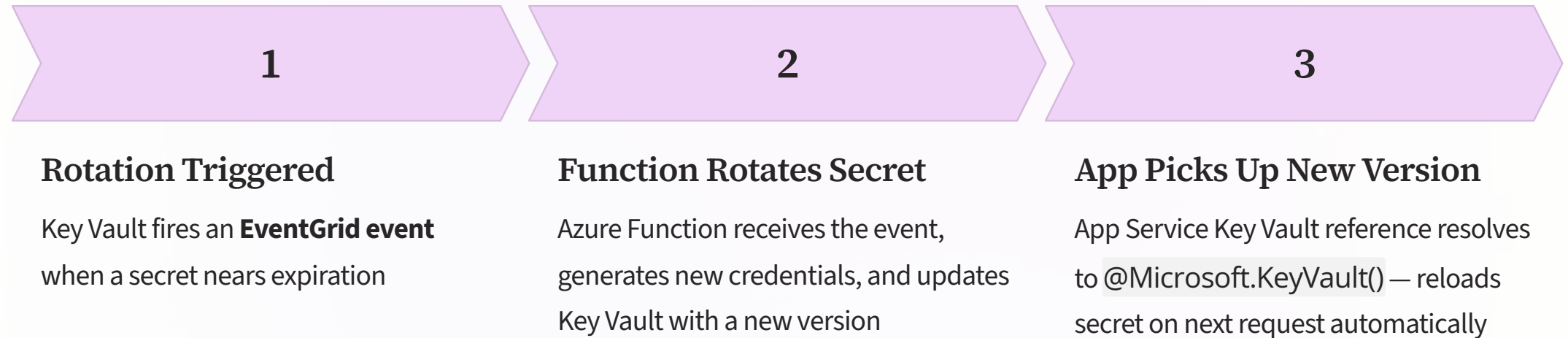
Real-World Scenario: Debugging a Distributed System



Incident: Intermittent 500 Errors in Production

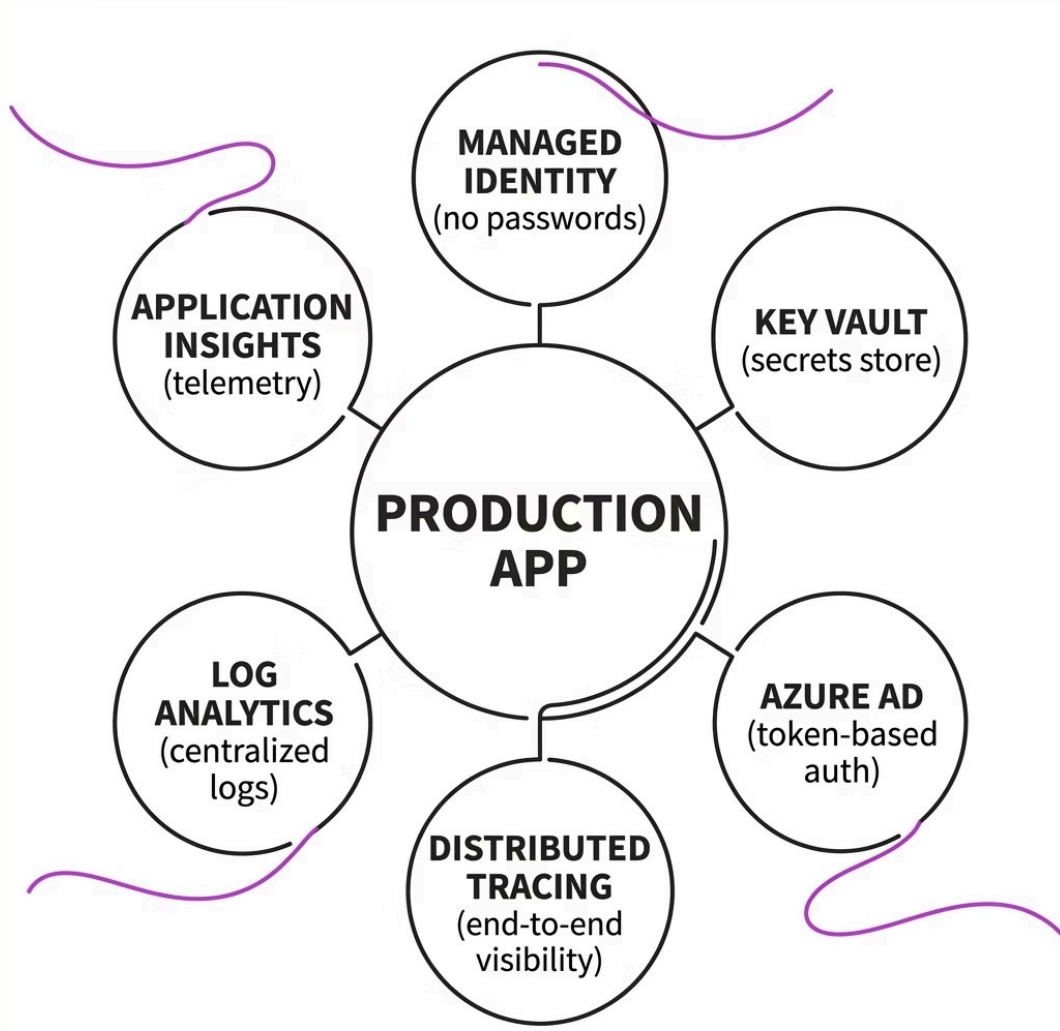
1. Alert fires: **failure rate > 5%** on API service
2. Open **Application Insights** → **Failures** blade — identify top exception type
3. Drill into a sample **end-to-end transaction** — see which dependency call failed
4. Check **Application Map** — downstream SQL database showing elevated latency
5. Run KQL query on **dependencies** table to confirm pattern
6. Root cause found: missing index on a query triggered by a recent deployment

Real-World Scenario: Automatic Secret Rotation



- ❏ Azure Storage and SQL Database support **native automatic rotation** via Key Vault — no custom Function needed. For custom credentials, the Event Grid + Function pattern is the recommended architecture.

Putting It All Together: Security + Observability



The Production-Ready Checklist

- ☒ No credentials in code or config files
- ☒ All secrets stored in Key Vault with RBAC
- ☒ App authenticates with Managed Identity
- ☒ APIs protected with Azure AD tokens
- ☒ Application Insights SDK instrumented
- ☒ Logs flowing to Log Analytics workspace
- ☒ Alerts configured on failure rates and latency

Key Vault References in App Service

How It Works

Instead of storing a plain value in App Service Application Settings, you store a **Key Vault reference**:

```
@Microsoft.KeyVault(  
  SecretUri=https://myvault.vault.azure.net/  
  secrets/ConnectionString/)
```

Azure resolves this at runtime — your app reads

```
Environment.GetEnvironmentVariable("ConnectionString"  
)
```

 and gets the decrypted value. No SDK calls needed in app code.

Why This Matters

- Works for App Service, Azure Functions, and Container Apps
- No code changes required in your application
- Secret updates automatically reflected without redeployment
- Requires system-assigned or user-assigned identity with **Secrets User** role

Exam Focus: Security & Monitoring

1

Managed Identity

Know system-assigned vs. user-assigned, and how to use `DefaultAzureCredential` for credential-free access across environments.

2

Key Vault Access

Understand RBAC roles (Secrets User vs. Officer), soft-delete behavior, versioning, and Key Vault references in App Service.

3

OAuth Flows

Know when to use Authorization Code + PKCE, Client Credentials, and On-Behalf-Of. Understand access vs. refresh token lifetimes.

4

Application Insights

Auto-collected telemetry types, connection string configuration, KQL basics, and what distributed tracing enables vs. standard logging.

DAY 5 LABS

Today's Lab Exercises

1

Secure App Using Managed Identity

2

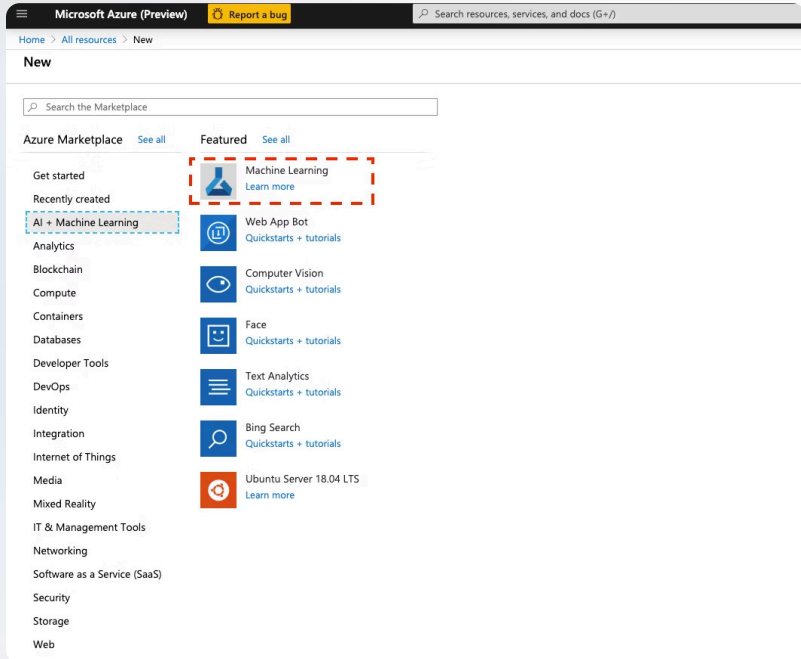
Use Azure Key Vault

3

Application Monitoring

4

Final Mini Project: Event-Driven File Processing System



Day 5 Complete — You're AZ-204 Ready

You've covered the full five-day curriculum. From compute and storage to messaging, APIs, security, and observability — you now have the skills to build, secure, and operate production-grade Azure applications.



Secure

Managed Identity + Key Vault



Authenticated

OAuth 2.0 + MSAL + Azure AD



Observable

App Insights + Log Analytics